

MedGraph

Guia de entendimento do projeto

Tech Challenge — Fase 3 · Pós-Tech 8IADT · Hospital Vida Plena (cenário fictício)

Este documento explica, em linguagem direta, o que o MedGraph é, como as peças se encaixam, o que cada parte do código faz e como qualquer pessoa que receba o repositório consegue executar o fine-tuning no Google Colab.

1. Do que se trata

O enunciado da Fase 3 pede um assistente virtual médico treinado com dados de um hospital, capaz de auxiliar em condutas clínicas, responder dúvidas do corpo médico e sugerir procedimentos com base em protocolos internos — coordenando fluxos de decisão automatizados e seguros.

O MedGraph atende a isso combinando três coisas que costumam aparecer separadas:

- uma **LLM ajustada** por fine-tuning para o formato de resposta que queremos;
- um **fluxo de decisão** explícito, com etapas nomeadas e auditáveis;
- **limites de atuação** verificados por código, e não apenas pedidos no prompt.

1.1 De onde vem o nome

Med de médico. **Graph** de grafo — a estrutura de dados que organiza o fluxo de decisão. O nome descreve a arquitetura: em vez de uma única chamada à LLM que faz tudo de uma vez, o projeto é um grafo de etapas nomeadas, cada uma com uma responsabilidade, ligadas por caminhos que podem se desviar.

É essa forma que torna o sistema auditável. Não se pergunta "por que a IA respondeu isso?", e sim "por quais nós esta consulta passou, e o que cada um decidiu?" — pergunta que tem resposta em arquivo.

1.2 O princípio que rege tudo

O assistente nunca prescreve. Ele apresenta evidência, aponta a fonte de cada afirmação e devolve a decisão ao médico responsável.

Isso não é postura de marketing: é o requisito 3 do enunciado, implementado nos guardrails, no nó de validação humana e nos testes automatizados. Quando o risco de uma resposta passa de um limiar configurado, a execução simplesmente **para** e espera um médico.

1.3 O que ele faz na prática

Um médico pergunta, opcionalmente vinculando um paciente:

"Qual a conduta antibiótica inicial para sepse de foco pulmonar neste paciente?"

O sistema então limpa a pergunta de dados identificáveis, descobre que tipo de pergunta é, consulta o prontuário estruturado, busca evidência científica e protocolos internos, pede à LLM uma resposta ancorada apenas nesse material, confere a resposta contra regras clínicas de segurança, exige citação de fonte, calcula um escore de risco e — se for o caso — retém a resposta para validação humana. Tudo isso fica registrado numa trilha de auditoria consultável.

No exemplo acima, o paciente tem alergia grave a penicilina. A resposta sugere uma cefalosporina. O sistema reconhece que **ceftriaxona é betalactâmico, a mesma classe da penicilina**, marca conflito crítico, e a execução para.

2. O vocabulário do projeto

Esta seção existe para que o resto do documento possa ser lido sem consultar nada por fora. Se você já conhece os termos, pule para a seção seguinte.

2.1 Sobre modelos de linguagem

LLM (Large Language Model). Um modelo de linguagem de grande porte — o tipo de programa por trás do ChatGPT. Ele foi treinado prevendo a próxima palavra em enormes quantidades de texto, e nesse processo acabou aprendendo gramática, fatos e padrões de raciocínio. Aqui usamos um modelo de **3 bilhões de parâmetros**, pequeno para os padrões atuais, que roda num computador comum.

Parâmetro. Cada um dos números internos do modelo, ajustados durante o treino. "3 bilhões de parâmetros" é o tamanho do modelo.

Token. A unidade em que o modelo enxerga o texto: um pedaço de palavra. Uma frase de 20 palavras costuma virar 25 a 30 tokens. Os custos e os limites de tamanho são todos contados em tokens.

Prompt. O texto que se envia ao modelo. Neste projeto os prompts ficam todos em `chains/prompts.py` — um prompt espalhado pelo código é um prompt que ninguém revisa.

Inferência. O ato de usar o modelo para gerar uma resposta, por oposição a treiná-lo.

2.2 Sobre o ajuste do modelo

Fine-tuning (ajuste fino). Pegar um modelo já treinado e continuar treinando com dados específicos, para que ele aprenda um comportamento novo. Não é ensinar medicina ao modelo: é ensinar o **formato** de resposta que queremos — decisão na primeira linha, citação da fonte no fim.

SFT (Supervised Fine-Tuning). O tipo de ajuste usado aqui: mostram-se pares de pergunta e resposta ideal, e o modelo aprende a produzir a segunda a partir da primeira.

Adapter. O arquivo que sai do treino: apenas as peças novas, sem o modelo inteiro. No nosso caso, cerca de 50 MB — pequeno o bastante para ficar versionado no Git ao lado do código que o produziu.

Época. Uma passada completa por todos os exemplos de treino.

Passo (step). Uma atualização dos parâmetros. Um treino tem centenas deles.

Perda (loss). O quanto o modelo está errando. É o número que precisa cair ao longo do treino; se a perda de treino cai e a de validação sobe, o modelo está decorando em vez de aprender.

2.3 Sobre a infraestrutura

GPU. A placa de vídeo, que faz as multiplicações de matriz em paralelo. Sem ela, treinar um modelo de 3 bilhões de parâmetros levaria dias.

VRAM. A memória da GPU. É o recurso escasso: determina se o treino cabe ou não.

CUDA. A tecnologia da NVIDIA que permite usar a GPU para cálculo. Bibliotecas de treino dependem dela — e é por isso que o fine-tuning não roda num Mac.

Google Colab. Um serviço do Google que dá acesso gratuito a uma GPU pelo navegador, com um limite diário de uso.

Hugging Face. O repositório público de onde se baixam modelos e conjuntos de dados.

Modelo gated. Modelo cujo download exige aceitar uma licença. O Llama 3.2 é um deles.

Ollama. Um programa que roda modelos de linguagem na sua própria máquina e os expõe como um serviço local. É o que serve o modelo depois de pronto, sem custo por consulta.

2.4 Sobre a recuperação de evidência

RAG (Retrieval-Augmented Generation). Geração aumentada por recuperação. Em vez de confiar no que o modelo memorizou, busca-se o material relevante primeiro e pede-se que ele responda **apenas** com base nesse material. É o que permite citar a fonte de cada afirmação — e o que reduz a chance de o modelo inventar.

Embedding. A representação de um texto como uma lista de números, construída de modo que textos com sentido parecido fiquem próximos. É o que permite buscar por significado, e não por palavra exata.

Vector store. O banco que guarda esses vetores e responde "quais textos mais se parecem com este?". Aqui usamos o **FAISS**, que roda local e é salvo em disco.

2.5 Sobre o fluxo e a segurança

LangChain. Biblioteca que padroniza a conversa com modelos de linguagem — montagem de prompt, chamada, tratamento da resposta.

LangGraph. Extensão do LangChain para fluxos em forma de grafo, com estado compartilhado, caminhos condicionais e possibilidade de **pausar a execução** à espera de uma pessoa. É essa última capacidade que sustenta a validação médica.

Guardrail. Uma verificação que limita o que o sistema pode fazer ou dizer. Aqui há um na entrada (limpa a pergunta e recusa o que está fora de escopo) e um na saída (exige citação, confere se as fontes existem, marca posologia).

PII (Personally Identifiable Information). Dados que identificam uma pessoa: nome, CPF, telefone, prontuário. O projeto os remove antes de qualquer texto chegar ao disco.

Trilha de auditoria. O registro, evento por evento, do que aconteceu em cada consulta — qual nó rodou, quanto demorou, o que decidiu. É o que permite reconstruir uma resposta meses depois.

3. Como funciona, em uma passagem

Uma pergunta atravessa um grafo de **14 nós**. Cada nó recebe um estado, devolve o estado modificado, e é automaticamente registrado na auditoria.

#	Nó	O que faz
1	guardrail_entrada	Remove identificadores e recusa pedidos fora de escopo
2	responder_recusa	Caminho alternativo quando a entrada é recusada
3	classificar_intencao	Dúvida clínica, consulta ao paciente, conduta, resumo...
4	consultar_prontuario	Alergias, medicações, exames, comorbidades
5	recuperar_evidencia	Busca semântica em artigos e protocolos
6	raciocinio_clinico	A LLM responde, ancorada no contexto recuperado
7	regras_clinicas	Alergias, interações, valores críticos, função renal
8	guardrail_saida	Exige citação, verifica fontes, marca posologia
9	reescrever	Nova tentativa quando a saída é reprovada
10	degradar_resposta	Resposta mínima segura quando as tentativas se esgotam
11	triagem_risco	Calcula o escore de risco da resposta
12	aguardar_validacao	Retém a resposta e espera um médico
13	emitir_alertas	Publica os alertas para a equipe
14	montar_resposta	Monta o texto final com fontes e disclaimer

Os nós 2, 9, 10 e 12 são desvios: só entram quando algo exige. O caminho feliz passa pelos demais.

3.1 Onde ficam as decisões de rota

O arquivo `grafo/rotas.py` concentra as quatro decisões condicionais do fluxo: depois do guardrail de entrada, depois da classificação, depois do guardrail de saída e depois da triagem de risco. Separá-las dos nós mantém cada nó com uma responsabilidade só.

4. Arquitetura em camadas



Cada dependência abaixo do grafo é substituível sem tocar no fluxo — o grafo conhece interfaces, não implementações.

4.1 Decisões técnicas e o porquê de cada uma

Decisão	Razão
Fine-tuning por QLoRA no Colab	9 GB de VRAM em vez de 48; adapter de 50 MB em vez de 6 GB
Modelo servido pelo Ollama	Roda offline, custo zero por consulta, código LangChain padrão
Embeddings locais (multilingual-e5-small)	Cobre inglês dos artigos e português dos protocolos, sem custo
FAISS	Vector store persistido em disco, sem serviço externo
Políticas em YAML	Governança clínica não deveria exigir leitura de Python
Auditoria por contextvars	Os nós mantêm assinatura limpa, sem carregar trace_id

5. Estrutura de pastas

fia_tech3/	
config/	
settings.py	Configuracao central, le o .env e valida
politicas.yaml	Limites de atuacao, em formato declarativo
src/medgraph/	
requisitos.py	Catalogo dos requisitos do enunciado
logging_config.py	Logging em tres destinos
auditoria.py	Trilha, trace por consulta, @instrumentar
dados/	Download, anonimizacao, curadoria, corpus
finetune/	Dataset de treino e utilidades do Colab
avaliacao/	Metricas, comparativos e graficos
llm/	Provedores de modelo e controle de custo
rag/	Indexacao vetorial e recuperacao com fontes
prontuario/	Acesso a base estruturada de pacientes
chains/	Pipelines LangChain
guardrails/	Guardrails de entrada e saida, regras clinicas
grafo/	Fluxo LangGraph
ui/	Painel Streamlit
data/	Dados brutos, processados, sinteticos, indices
models/	Adapter LoRA e modelo quantizado
notebooks/colab/	Notebooks de fine-tuning e exportacao
scripts/	Pontos de entrada de linha de comando
tests/	Testes automatizados
docs/	Relatorio, diagramas, graficos, rastreabilidade
logs/	Trilha de auditoria e traces

6. O que cada módulo faz

6.1 Fundação

`config/settings.py` — classe `Settings`, baseada em `Pydantic`. Lê o `.env`, valida os valores e deriva todos os caminhos do projeto. Um erro de configuração falha aqui, na largada, e não vinte minutos depois.

`requisitos.py` — classe `Requisito` e o `CATALOGO` com os 13 requisitos do enunciado. Cada docstring do código cita a tag correspondente, e a matriz de rastreabilidade é gerada varrendo essas tags.

`logging_config.py` — classes `FormatadorJSONL` e `FiltroApenasAuditoria`. Configura três destinos simultâneos: console colorido, arquivo de texto rotativo, e a trilha JSONL formal.

`auditoria.py` — o coração da rastreabilidade. Classes `TipoEvento`, `Desfecho`, `EventoAuditoria` e `TrilhaAuditoria`. O decorador `@instrumentar` registra automaticamente início, fim, duração e o delta de estado de cada nó. A alternativa — repetir blocos de log em cada nó — falharia no primeiro esquecimento, e um nó sem rastro invalidaria a garantia de auditabilidade.

6.2 Dados

`dados/anonimizador.py` — classes `TipoPII`, `Politica`, `Achado` e `Anonimizador`. Detecta e trata dados pessoais. Cuidado central do projeto: um anonimizador que apaga valor laboratorial entrega dado limpo e clinicamente inútil.

`dados/baixar_pubmedqa.py` — obtém o PubMedQA do Hugging Face.

`dados/curadoria.py` — classe `RelatorioCuradoria` e as funções `higienizar`, `curar`, `dividir_estratificado` e `balancear_por_rotulo`. A divisão estratificada preserva a proporção das classes; o balanceamento corrige a classe rara.

`dados/construir_banco.py` — monta a base SQLite de prontuários sintéticos.

6.3 Fine-tuning

`finetune/preparar_dataset_sft.py` — monta o dataset supervisionado a partir do PubMedQA curado, dos protocolos e do FAQ.

`finetune/colab_utils.py` — tudo o que o notebook do Colab precisa, em módulo Python testável: `verificar_gpu`, `config_quantizacao` (a parte "Q" do QLoRA, em NF4 com dupla quantização), `config_lora`, `montar_configuracao_sft`, `montar_treinador`, `alinhar_precisao_dos_adaptadores`, `grafico_de_perda` e `salvar_metadados` (o cartão de treino — sem ele o adapter seria um binário sem procedência).

O notebook é deliberadamente magro: ele orquestra, não implementa. Lógica dentro de célula de notebook não tem teste, não tem lint e não aparece em diff legível.

6.4 Modelo e custo

`llm/provider.py` — classes `CallbackCusto` e `ChatEco`, funções `obter_llm` e `obter_llm_com_fallback`. Abstrai qual modelo responde: o ajustado no Ollama, a API paga, ou um eco determinístico para testes.

`llm/custo.py` — classes `RegistroUso`, `ContadorCusto` e `OrcamentoExcedidoError`. Contabiliza tokens e dólares por chamada e **bloqueia** chamadas pagas ao atingir o teto da sessão.

6.5 Recuperação de evidência

`rag/indexar.py` — classes `EstatisticasIndice` e `EmbeddingsComPrefixo`. Constrói o índice FAISS. O prefixo existe porque os modelos E5 exigem marcar o texto como consulta ou como documento.

`rag/recuperador.py` — classes `Trecho` e `Recuperador`. Devolve trechos já etiquetados com o marcador de fonte, o que torna a citação verificável.

6.6 Prontuário

`prontuario/modelos.py` — as entidades do domínio: `Paciente`, `Alergia`, `Medicacao`, `Exame`, `Comorbidade`, `SinalVital`, `Evolucao`.

`prontuario/repositorio.py` — classe `RepositorioProntuarios`, a única porta de acesso à base estruturada.

6.7 Pipelines LangChain

`chains/prompts.py` — todos os textos de prompt num lugar só. Um prompt espalhado pelo código é um prompt que ninguém revisa.

`chains/chain_triagem.py` — classe `ResultadoTriagem`. Classifica a intenção da pergunta.

`chains/chain_rag.py` — monta a mensagem com o contexto recuperado e obtém a resposta ancorada.

6.8 Segurança e validação

`guardrails/politicas.py` — classes `PadraoBloqueio` e `Politicas`. Carrega o `politicas.yaml` e normaliza o texto antes de comparar, para que acentuação não contorne um bloqueio.

`guardrails/entrada.py` — classe `ResultadoEntrada`. Limpa a pergunta e recusa o que está fora de escopo.

`guardrails/regras_clinicas.py` — o módulo mais denso do projeto. Classes `Interacao`, `Severidade`, `Achado` e `ResultadoVerificacao`; verificações de alergias, interações medicamentosas, valores laboratoriais críticos, função renal e populações especiais. A função `em_contexto_de_evitacao` distingue "evitar penicilina" de "prescrever penicilina" — sem ela, o sistema alertava justamente quando o assistente acertava.

`guardrails/saida.py` — classes `Falha` e `ResultadoSaida`. Exige citação de fonte, verifica se as fontes citadas existem, marca posologia e acrescenta o disclaimer.

6.9 O grafo

`grafo/estado.py` — `EstadoClinico`, o dicionário tipado que atravessa todos os nós.

`grafo/nos.py` — as 14 funções de nó, cada uma decorada com `@instrumentar`.

`grafo/rotas.py` — as quatro decisões condicionais.

`grafo/construir.py` — monta e compila o grafo.

`grafo/executar.py` — classe `Consulta` e as funções `consultar`, `validar` e `consultas_pendentes`. É a fachada que o painel e os scripts usam.

`grafo/diagrama.py` — gera o diagrama do fluxo em PNG, Mermaid e ASCII.

6.10 Avaliação e interface

`avaliacao/metricas.py` — classe `ResultadoAvaliacao`, extração da decisão, baseline da classe majoritária e a tabela comparativa.

`avaliacao/avaliar.py` e `avaliacao/graficos.py` — executam o comparativo entre os sistemas e desenharam os gráficos.

`ui/app_streamlit.py` e `ui/componentes.py` — o painel visual, com abas para consulta, prontuário, fontes, alertas, trilha do grafo e validação médica.

7. O fine-tuning explicado

Esta é a parte do projeto que costuma parecer mais opaca, e ela tem uma lógica simples por trás.

7.1 O problema

Queremos que o modelo responda **sempre no mesmo formato**: a decisão na primeira linha, o raciocínio em seguida, a citação da fonte no fim. Um modelo genérico responde bem, mas cada vez de um jeito — e um formato instável quebra tudo o que vem depois, porque os guardrails precisam encontrar a citação para verificá-la.

Ajustar o modelo resolve isso. Mas treinar um modelo de 3 bilhões de parâmetros do jeito tradicional exige guardar, além dos pesos, os gradientes e os estados do otimizador: cerca de **48 GB de memória de vídeo**. Nenhuma GPU gratuita chega perto disso — a T4 do Colab tem 16 GB.

7.2 A ideia do LoRA

LoRA significa *Low-Rank Adaptation*, adaptação de baixo posto. A intuição é esta: para ensinar um formato de resposta a um modelo que já sabe português e já leu literatura médica, não é preciso mexer nos 3 bilhões de números. A mudança necessária é pequena e tem estrutura.

Então congela-se o modelo inteiro e adicionam-se, ao lado de certas camadas, **duas matrizes finas** cujo produto representa o ajuste. Treinam-se apenas elas — cerca de **24 milhões de parâmetros, 0,7% do total**.

	Fine-tuning completo	LoRA
Parâmetros treinados	3,2 bilhões	24 milhões
Memória necessária	~48 GB	~16 GB
Tamanho do resultado	~6 GB	~50 MB

Os 50 MB são a razão de o resultado do treino caber no Git, versionado junto com o código que o produziu.

7.3 O "Q" de QLoRA

Falta um passo. Mesmo congelado, o modelo base precisa estar na memória — e em precisão normal ele ocupa ~6 GB, o que ainda aperta.

Quantização é reduzir a precisão numérica dos pesos. Em vez de 16 bits por número, usam-se **4 bits**. O modelo cai de ~6 GB para ~2 GB, com perda de qualidade pequena.

O formato usado é o **NF4** (*NormalFloat4*), proposto no artigo do QLoRA: otimizado para números que seguem distribuição normal, que é o caso dos pesos de uma rede treinada. Perde menos que o int4 comum. Há ainda a **dupla quantização**, que comprime as próprias constantes de quantização e economiza mais uns 0,4 bit por parâmetro.

QLoRA = Quantized LoRA: o modelo base entra quantizado em 4 bits e congelado; os adaptadores treinam por cima, em precisão maior. O cálculo acontece numa precisão mais alta do que o armazenamento — os pesos são desquantizados no momento da multiplicação.

É exatamente por isso que o treino é lento numa GPU modesta: cada multiplicação de matriz paga o custo de desquantizar antes de calcular. Não é defeito de configuração; é o preço de caber.

7.4 Onde os adaptadores são aplicados

Nas **sete projeções** de cada bloco do modelo: as quatro da atenção (q , k , v , o) e as três do MLP ($gate$, up , $down$).

Muitos tutoriais aplicam LoRA apenas em q_proj e v_proj . Isso rende menos quando a tarefa muda o **estilo** da resposta — e é o nosso caso: queremos que o modelo passe a responder num formato rígido.

7.5 O que sai do treino

Um diretório com o adapter, o tokenizador, a curva de perda e o `cartao_de_treino.json`. Esse cartão guarda qual modelo base foi usado, com quais hiperparâmetros, em qual GPU, com quais versões de biblioteca, em quanto tempo e com que perda final. **Sem ele, o adapter seria um binário sem procedência** — e não haveria como responder, meses depois, como aquele resultado foi obtido.

7.6 Do adapter ao modelo que responde

O adapter sozinho não é utilizável por um servidor comum. Faltam três passos, que o segundo notebook executa:

Fusão (merge). Somar as matrizes do adapter aos pesos do modelo base, produzindo um modelo único e completo. É aqui que importa fundir na arquitetura **certa**: um adapter treinado sobre um modelo e fundido em outro não falha — só responde pior, silenciosamente.

Conversão para GGUF. O GGUF é o formato de arquivo que o `llama.cpp` e o Ollama entendem, desenhado para rodar modelos em hardware comum. A conversão também quantiza de novo, agora em **Q4_K_M** — um nível de compressão que equilibra tamanho e qualidade.

Publicação e registro. O arquivo vai para o Hugging Face, e na máquina local o Ollama o registra sob um nome. A partir daí, `OLLAMA_MODEL=medgraph` no `.env` faz todo o restante do projeto passar a conversar com o modelo ajustado — sem mudar uma linha de código.

7.7 Um detalhe que importa: o chat template

Cada família de modelos marca o início e o fim de cada fala com tokens especiais próprios. Esse padrão chama-se **chat template**.

O `Modelfile` do Ollama, neste projeto, **não fixa** um template: usa o que vem embutido no próprio GGUF. Se ele trouxesse o formato de um modelo escrito à mão — como chegou a trazer numa versão inicial —, trocar de modelo base degradaria as respostas em silêncio, porque o modelo veria

uma sequência de tokens diferente da que aprendeu.

8. Como executar na sua máquina

```
git clone https://github.com/alexandreccarmo/fia_tech3.git
cd fia_tech3
make setup
```

O `make setup` cria o ambiente virtual, instala as dependências, gera o `.env` a partir do `.env.example` e roda os testes.

Requisitos: Python 3.12 (o stack de ML ainda não tem wheels para 3.13/3.14), Ollama, e cerca de 12 GB de disco.

8.1 Comandos principais

Comando	O que faz
<code>make ajuda</code>	Lista todos os comandos
<code>make ambiente</code>	Diagnóstico item a item do ambiente
<code>make testes</code>	Roda a suíte
<code>make dados</code>	Baixa, anonimiza e cura os dados
<code>make indexar</code>	Constrói o índice FAISS
<code>make grafo</code>	Demonstração do fluxo no terminal
<code>make avaliar</code>	Avaliação comparativa e gráficos
<code>make app</code>	Abre o painel Streamlit
<code>make relatorio</code>	Regenera o relatório técnico

Para passar opções a um comando, use o separador `--`:

```
make grafo -- --diagrama
make avaliar -- --rapido
make modelo -- --ajustado
```

9. O Colab: como o link funciona

Esta é a parte que mais confunde, e a explicação é mais simples do que parece. **Não há geração de nada.** São três mecanismos independentes.

9.1 O notebook é um arquivo do repositório

`notebooks/colab/01_finetune_qlora_pubmedqa.ipynb` é um arquivo versionado no Git como qualquer outro. Um `.ipynb` é apenas JSON. Ele não é produzido por nenhum processo: foi escrito, commitado e enviado.

9.2 O link é uma convenção de URL

O Colab tem um carregador de GitHub que funciona por padrão de endereço. Compare:

```
https://github.com/USUARIO/REPO/blob/BRANCH/CAMINHO.ipynb
https://colab.research.google.com/github/USUARIO/REPO/blob/BRANCH/CAMINHO.ipynb
```

É a mesma URL, trocando `github.com/` por `colab.research.google.com/github/`. O Colab baixa aquele JSON e o abre no editor. Não existe integração configurada, nem token, nem aplicativo instalado.

Funciona sem login porque o repositório é público. Se fosse privado, o Colab pediria autorização de acesso ao GitHub.

9.3 O código chega lá por git clone

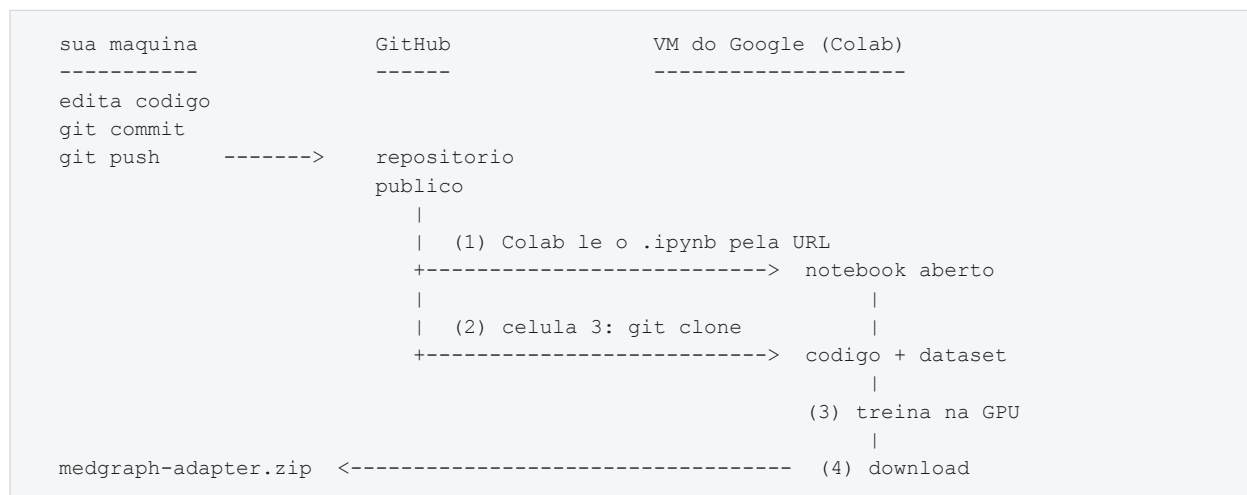
O notebook sozinho não traz o código do projeto. Quem traz é a terceira célula:

```
if not os.path.isdir("/content/fia_tech3"):
    !git clone --depth 1 https://github.com/alexandreccarmo/fia_tech3.git /content/fia_tech3

os.chdir("/content/fia_tech3")
sys.path.insert(0, "/content/fia_tech3/src")
```

Ela baixa o repositório inteiro para dentro da máquina virtual do Google e coloca `src/` no caminho de importação. É por isso que a célula seguinte consegue fazer `from medgraph.finetune import colab_utils` — esse é o código do projeto. O dataset de treino vem junto no clone, porque também está versionado.

9.4 O fluxo completo



O caminho é de mão única: o Colab lê do GitHub e nunca escreve de volta sozinho. O resultado do treino retorna como download manual do arquivo .zip.

10. Passando o projeto para outra pessoa

Um terceiro que receba este repositório consegue gerar o próprio link do Colab em dois passos.

10.1 Se for usar o repositório original

Basta abrir o link, sem precisar de nada:

```
https://colab.research.google.com/github/alexandreccarmo/fia_tech3/blob/main/notebooks/colab/01_finetune_qlora_pubmedqa.ipynb
```

Funciona para qualquer pessoa, porque o repositório é público. É o caminho recomendado para quem só quer executar.

10.2 Se for trabalhar sobre uma cópia própria

Quem for modificar o projeto precisa que a célula 3 clone o **repositório dele**, e não o original — senão o Colab treinaria com o código de outra pessoa.

Passo 1. Publique a cópia no GitHub, como repositório **público**:

```
https://github.com/NOVO_USUARIO/NOVO_REPO
```

Passo 2. Edite a variável `REPO` na terceira célula do notebook para apontar para a cópia:

```
REPO = "https://github.com/NOVO_USUARIO/NOVO_REPO.git"
```

E ajuste o caminho do clone, se mudar o nome do diretório.

Passo 3. Monte o link trocando as três partes do endereço:

```
https://colab.research.google.com/github/NOVO_USUARIO/NOVO_REPO/blob/main/notebooks/colab/01_finetune_qlora_pubmedqa.ipynb
```

Passo 4. Faça o mesmo para o segundo notebook, `02_exportar_gguf.ipynb`.

10.3 Atalho pelo próprio Colab

Também é possível chegar lá sem montar a URL à mão. Dentro do Colab:

Arquivo → **Abrir notebook** → **aba GitHub**, digite `usuario/repositorio` e escolha o notebook na lista.

10.4 Se o repositório for privado

Na mesma aba GitHub do Colab existe a opção **Incluir repositórios privados**. Ela pede autorização da conta do GitHub. O link direto por URL, porém, deixa de funcionar para quem não tiver acesso — e a célula do `git clone` também falharia, já que roda sem credenciais dentro da VM.

Por isso o projeto é público: é o que torna "treinar" equivalente a "abrir um link".

10.5 O que a pessoa precisa ter

- Conta Google, para o Colab
- Conta no Hugging Face, para baixar o modelo base
- Aceite da licença do modelo, quando ele for *gated* (o caso do Llama 3.2)

O aceite é vinculado à conta do Hugging Face, não à conta Google. Trocar de conta Google não faz o aceite deixar de valer.

11. Requisitos do enunciado

O projeto cataloga cada exigência com um código, citado nas docstrings do código que a atende. A matriz completa é gerada automaticamente por `make rastreabilidade`.

Código	Exigência
REQ-1	Fine-tuning de um modelo LLM com dados do hospital
REQ-1a	Preparo dos dados: preprocessing, anonimização e curadoria
REQ-2	Pipeline LangChain integrando a LLM customizada
REQ-2a	Consultas a base de dados estruturada
REQ-2b	Respostas contextualizadas com dados do paciente
REQ-3a	Limites de atuação para evitar sugestões impróprias
REQ-3b	Logging detalhado para rastreamento e auditoria
REQ-3c	Explainability: indicar a fonte da informação
REQ-4	Projeto modularizado com instruções completas no README
REQ-E1	Código-fonte com os fluxos do LangGraph
REQ-E2	Dataset anonimizado ou dados sintéticos
REQ-E3	Relatório técnico detalhado
REQ-E4	Vídeo de até 15 minutos

12. Testes e honestidade dos números

A suíte roda em poucos segundos e cobre configuração, catálogo de requisitos, logging, auditoria, cálculo de custo, trava de orçamento, regras clínicas e a consistência do arquivo de políticas — incluindo a verificação de que conduta terapêutica **sempre** exige validação humana.

Há ainda uma categoria de teste incomum: `tests/test_documentacao.py` verifica as afirmações numéricas da documentação contra o código. Ele nasceu de um erro real — o grafo foi planejado com doze nós, ganhou mais dois durante a implementação, e cinco arquivos continuaram afirmando "doze". Nenhum teste falhou, nenhum linter reclamou, e o erro sobreviveu a várias revisões porque documentação não é executada.

O relatório técnico segue o mesmo princípio: a narrativa fica em `docs/relatorio_base.md` e os números são lidos dos artefatos que o pipeline produziu. Um relatório com números digitados à mão começa correto e envelhece errado.

13. Nota de transparência

Nenhum dado real de paciente é utilizado. Prontuários, protocolos e documentos do Hospital Vida Plena são gerados por script, com nomes fictícios. Ainda assim, o pipeline de anonimização é aplicado sobre eles, demonstrando a técnica exigida no enunciado e garantindo que o mesmo código funcionaria sobre dados reais.

Este é um projeto acadêmico. Não foi submetido a comitê de ética, não passou por validação clínica e não deve ser utilizado em assistência a pacientes.